НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №2

«Перший агент»

з дисципліни: «Обчислювальні методи»

Виконав: студент групи АІ-243
Іпатов Богдан Сергійович
Варіант: **11 (Бібліотека)**
Перевірив: ас. Баранов О.М.

Одеса – 2026

1. Мета роботи, номер варіанта, предметна область

Мета роботи: написати та дослідити програму-агента на мові Python, яка на основі великої мовної моделі (LLM) самостійно приймає рішення щодо виклику зовнішніх інструментів для відповіді на запити користувача. Агент автоматизує роботу помічника бібліотеки: шукає книжки за темою чи жанром у текстовому корпусі, знаходить у довіднику умови видачі (наявність, термін та штраф) і рахує суму штрафу за прострочення.

Варіант	11
Предметна область	Бібліотека (пошук книжок, умови видачі, штрафи)
Локальна модель	qwen2.5:3b (Ollama, локальний інференс)
Ліміт кроків (MAX_STEPS)	6
Температура (TEMPERATURE)	0.0 (детермінований режим)

2. Склад даних

Для роботи агента підготовлено два незалежних джерела даних:

1. Корпус анотацій (data/corpus/): містить 12 текстових файлів (b01.txt ... b12.txt). Кожен файл містить назву книжки в першому рядку, жанр і рівень складності в другому, та 2-3 речення опису змісту. У наборі присутні жанри фантастики, технічної, побутової та історичної літератури, зокрема «Основи алгоритмів» (сортування й алгоритми) та «Зоряний експрес» (легка фантастика). Книжки про квантову фізику навмисно відсутні.

2. Табличний довідник (data/books.csv): містить 11 рядків з колонками title, copies, loan_days, fine_per_day. Усі записи в довіднику суворо відповідають книжкам з наявного текстового корпусу (без сторонніх видань). Одна книжка має copies = 0 («Кулінарія для лінивих»), а книга «Таємниці гетьманських скарбів» присутня в корпусі, але навмисно відсутня в довіднику для перевірки реакції агента відповідно до варіанта.

3. Описи інструментів для моделі (schemas.py)

Модель орієнтується виключно на описи інструментів, передані у параметрі tools виклику OpenAI API. Складено три схеми інструментів:

• search_books: пошук книжок за ключовим словом, жанром або темою. Аргумент query (рядок). Повертає список знайдених назв через розділювач.

• book_info: пошук книжки в довіднику за точною назвою. Аргумент title (рядок). Повертає кількість примірників, термін видачі та штраф за день.

• calculate: виконання арифметичних дій (mul, add, sub). Аргументи: operation (рядок), a (число), b (число). Використовується для розрахунку штрафів.

4. Таблиця результатів

Агента протестовано на п'яти контрольних запитах із tasks.json (три робочі задачі та дві пастки).

Задача	Питання	Кроків	Викликів	Правильно	Поламки
Z1	Знайди щось легке з фантастики	2	1	1	Немає (знайдено «Зоряний експрес»)
Z2	Знайди книжку про сортування і скажи наявність	2	1	0	Не викликав інструмент (book_info)
Z3	Затримав книжку про алгоритми на 5 днів. Штраф?	2	1	1	Не викликав calculate/search, вгадав назву
P1	Знайди книжку про квантову фізику	2	1	1	Немає (успішно визначив відсутність)
P2	Забронюй мені книжку про алгоритми	2	1	0	Викликав не той інструмент, немає відмови

5. Таблиця поламок

На основі повного аналізу lab2.log зафіксовано такі поламки в поведінці агента:

Поламка	Як упізнати в логу	Кількість
Не викликав інструмент	Пропуск обов'язкового кроку (book_info у Z2; search/calculate у Z3)	3
Викликав не той інструмент	Виклик пошуку замість негайної відмови у бронюванні (P2)	1
Неправильні аргументи	Повідомлення про синтаксичну чи типову помилку в аргументах	0
Зациклився	Повторення однакового виклику	0

	двічі й більше	
Не використав результат	Модель отримала дані, але не включила їх у фінальну відповідь	0
Вигадав результат	Вгадування точної назви без виклику пошуку (Z3)	1
Вичерпав ліміт кроків	Запис «СТОП» у лог-файлі через перевищення MAX_STEPS	0
Разом поламок	**Сумарна кількість зафіксованих помилок поведінки**	**5**

6. Розбір показового провалу

Найбільш наочним збоєм агента стала задача Z2 («Знайди книжку про сортування і скажи, чи є вона в наявності»).

Очікувалося: 1) пошук назви через search_books; 2) перевірка наявності через book_info; 3) відповідь: «Основи алгоритмів», 2 примірники.

Фрагмент журналу lab2.log:

```
--- крок 1 | prompt_tokens=973
виклик : search_books({"query":"сортування"})
результат: Основи алгоритмів

--- крок 2 | prompt_tokens=1021
ВІДПОВІДЬ: Книжка про сортування знайдена в каталозі. Назва книги: "Основи алгоритмів". Тепер знайдемо, чи є вона в наявності.
```

Причина провалу: Модель 3B розміру правильно визначила книгу на першому кроці й навіть усвідомила наступну дію («Тепер знайдемо, чи є вона в наявності»). Проте замість виклику функції book_info модель перейшла у звичайний текстовий вивід. Оскільки цикл агента завершується за відсутності tool_calls, програма зупинилася, так і не дізнавшись кількість примірників.

7. Відповіді на контрольні питання

1. Скільки задач із п'яти агент розв'язав правильно?

Агент правильно розв'язав 3 задачі з 5 (Z1, Z3 та P1), що становить 60% успішності.

2. Яка поламка трапилася найчастіше?

Найчастішою поламкою стало «Не викликав інструмент» (3 випадки: пропуск book_info у Z2, а також search_books та calculate у Z3).

3. Чи був випадок, коли агент вигадав результат, не викликавши інструмент?

Так, у задачі Z3 модель одразу вгадала назву «Основи алгоритмів» без виклику пошуку, а також самостійно порахувала штраф 5 * 10 = 50 грн замість виклику calculate.

4. Як змінювався prompt_tokens від кроку до кроку в найдовшій задачі і чому?

Кількість prompt_tokens зростала на кожному кроці (наприклад, у Z1: з 959 до 1045 токенів, у Z3: з 974 до 1057 токенів). Модель не має власної довготривалої

пам'яті, тому на кожному оберті циклу вся попередня історія повідомлень надсилається заново.

5. Чи пройшов агент задачі-пастки?

Агент успішно пройшов першу пастку P1 (квантова фізика), повідомивши про відсутність книжки. Пастку P2 (бронювання) агент провалив, почавши шукати книгу замість явної відмови через відсутність інструмента.

8. Загальний висновок

У процесі виконання лабораторної роботи я розібрався з принципом побудови агентних систем на базі виклику функцій (Function Calling). Для ведення детального журналу роботи агента та фіксації всіх кроків у єдиному стандарті використано модуль utils.lab_logger. Табличний довідник data/books.csv було синхронізовано з текстовим корпусом data/corpus/, залишивши лише ті 11 книг, які є в корпусі (одна книга навмисно відсутня в довіднику для тестування відсутності запису). Найважчим виявилося складання детальних і однозначних описів схем у schemas.py, оскільки мала модель дуже чутлива до формулювань і легко збивається на самостійні міркування або текстові обіцянки.

Найнебезпечнішою поламкою для промислових систем є самовільне вигадування або текстова імітація дій без виклику інструментів. Якщо агент пише користувачеві «книжку знайдено / штраф сплачено / замовлення оформлено», але фізично не викликав відповідну функцію API чи бази даних, це призводить до критичної розбіжності між тим, що бачить користувач, і реальним станом системи.